Alpha Evolve

Evolutionary Code Optimization Through Collaborative AI Agent Work Sessions

---

1. Why This System Exists

1.1 The Problem

Optimizing code is an iterative process. A developer writes a solution, tests it, sees where it falls short, tries a different approach, tests again. The best solutions often emerge not from a single brilliant insight but from the accumulation of many small discoveries: this data structure works better here, this edge case needs special handling, this optimization technique helps for large inputs but hurts for small ones.

Large language models are good at each individual step — writing code, analyzing results, proposing improvements. But they lack persistence. Each conversation starts fresh. There is no memory of what was tried, what worked, what failed, or why. The same dead ends get explored repeatedly. Promising directions get abandoned because no one remembered them.

The original AlphaEvolve system by Google DeepMind demonstrated that wrapping an LLM in an evolutionary loop — where solutions are stored, scored, and fed back as context for generating better solutions — can lead to genuine discoveries, including improvements to 56-year-old mathematical algorithms. But it operates as a single LLM pipeline: one prompt sampler, one model, one loop.

This system asks: what if instead of one LLM doing everything, we had a team of specialized agents, each with a distinct cognitive role, sharing knowledge through a structured file system, coordinating through an architect, and learning not just about the problem but about their own process?

1.2 The Core Idea

Alpha Evolve decomposes the evolutionary optimization process into specialized agent roles, each implemented as a Claude Code work session. Every agent can read files, write code, execute evaluation scripts, see results, iterate, and submit its best work. The agents share a common file system containing solutions, accumulated knowledge, and feedback. An Architect agent coordinates each generation by writing short briefs that guide each agent's attention. After each round, analysis agents extract knowledge from results and critique the system itself.

The file system is the single source of truth. The orchestrator — a simple Python loop — stores nothing in memory. It reads files to know what to do, launches agents by pointing them to their role definition and brief, and moves output files to their permanent locations between phases. If the orchestrator crashes and restarts, it reconstructs its state entirely from what files exist.

The result is a system where:

- Multiple approaches are explored in parallel by agents with different strategic roles
- Knowledge accumulates across generations in a hierarchical, compressed form that scales to many generations
- Every piece of knowledge tracks its provenance — who discovered it, when, based on what evidence
- Agents read knowledge at the right level of detail — high-level summaries for orientation, full detail only when relevant
- A dedicated Experimentator runs controlled tests that produce knowledge rather than solutions
- Agents report what they lacked, enabling the system to improve its own context quality
- A dedicated System Critic identifies pipeline problems and missing capabilities
- All system state lives in files — the orchestrator is stateless and recoverable
- The user can observe, intervene, or extend the system at any point by editing files

1.3 What Problems It Solves

The system targets optimization problems where candidate solutions can be automatically evaluated — sorting algorithms, scheduling heuristics, mathematical constructions, ML hyperparameter configurations, code optimization. The user provides a problem description, constraints, an evaluation script, and test cases. The system evolves solutions toward a target score.

Beyond the immediate optimization task, the system also solves meta-problems that plague LLM-based optimization: context loss between iterations, repeated exploration of dead ends, inability to learn from failures, lack of strategic coordination between parallel search efforts, and — critically — knowledge base bloat that overwhelms agent context windows as generations accumulate.

---

2. Design Philosophy

2.1 Agents Are Work Sessions, Not Function Calls

Each agent runs as a full Claude Code session with tool access. It reads files, writes code, runs evaluation scripts, sees results, and iterates — all within a single session. An agent might try five different approaches, evaluate each, and submit only its best. This is fundamentally different from systems where an LLM produces a single output that gets evaluated externally. The agent gets immediate feedback and can learn within its session.

2.2 Files Are the Single Source of Truth

All system state lives in the file system. The orchestrator is a stateless Python loop that reads files to determine what to do next and launches agents by pointing them to files. Agents read from the shared file system and write only to their designated output directories. The orchestrator moves outputs to permanent locations between phases.

This means: the Architect controls strategy by writing files (briefs, manifest). The Evaluator updates knowledge by writing files. The orchestrator just reads those files and acts. If any process crashes, the system can resume by examining which files exist. No state is held in memory, in databases, or in message queues. Files are the API between every component.

2.3 Read Broadly, Write Narrowly

All agents can read the entire project file system. Reading is unrestricted because agents need broad context to make good decisions — an Explore agent might study a top solution in detail because something in a cluster summary triggered its curiosity.

Writing is restricted. Every agent writes only to its own `output/` directory inside its workspace. The orchestrator is the only process that moves files from agent output directories to their permanent locations in the shared file system. This means:

- An agent cannot accidentally overwrite another agent's work
- An agent cannot corrupt the knowledge base, population, or briefs
- Every file in the shared system was placed there by the orchestrator
- The user can inspect exactly what each agent produced before the orchestrator moved it
- Analysis agents (Evaluator, Consistency Reviewer) write to designated output directories that the orchestrator copies to knowledge and feedback locations

The Experimentator agent runs code in a sandboxed subdirectory within its workspace, further isolating its test scripts from the rest of the system.

2.4 Information Flows Up as Compression, Down as Curation

Raw tries and metrics flow upward through the Evaluator agent, which compresses them into structured knowledge: ideas, patterns, and facts. That knowledge is further compressed into topic cluster summaries and a single State of Affairs document. The Architect reads the compressed layers and curates them into per-agent briefs — short reading lists that guide attention. No one assembles giant context blocks. Agents read at the level of detail they need: summaries for orientation, full files only when directly relevant.

2.5 Knowledge Has Three Layers of Resolution

The knowledge system is organized as a three-layer hierarchy. Layer 0 is a single State of Affairs document — a short narrative covering where the system stands, what works, what has been tried, and what the current frontier is. Layer 1 is a set of topic cluster summaries that group related ideas into themes, each with aggregated evidence and status. Layer 2 is the full detail: individual idea files, pattern files, fact files, each with complete provenance and evidence.

Every agent reads Layer 0. Agents read the Layer 1 clusters relevant to their task. Agents drill into Layer 2 files only when a specific idea or pattern is directly relevant to what they are working on. This ensures that an agent in generation 50 gets the same quality of orientation as an agent in generation 3, without drowning in accumulated detail.

2.6 Every Piece of Knowledge Has Provenance

Every idea, pattern, fact, and observation records who created it, when, with what certainty, based on what evidence, and when it was last confirmed to still be true. An insight from generation 3 that was last confirmed in generation 3 is suspicious by generation 10. An insight confirmed in generation 9 is solid. Provenance enables the Consistency Reviewer to audit knowledge systematically and enables agents to weigh claims appropriately.

2.7 Structured Shell, Freeform Payload

The data model uses YAML frontmatter for structured metadata (queried by the orchestrator for filtering, aggregation, and ranking) wrapped around a markdown body of freeform text (read by agents for comprehension). The orchestrator uses the shell. Agents read the text. This balances machine-queryability with the nuanced reasoning that LLMs excel at.

2.8 The System Debugs Itself

Every agent gets debriefed after its run: what did you lack, what might be wrong, what would you do differently? These reports feed back to the Architect (for better briefs), the Evaluator (for knowledge correction), and the System Critic (for pipeline improvements). The System Critic looks at the system itself — not the solutions — and identifies missing capabilities, prompt problems, and actionable recommendations for the user. The system is aware of its own limitations and reports them.

2.9 Debunked Knowledge Is Still Knowledge

When an idea is proven wrong, it is marked as debunked with an explanation of why. It is not deleted. Debunked ideas prevent agents from rediscovering dead ends. But if an agent independently tries a debunked approach and succeeds, that is a legitimate discovery — it means the debunking was context-dependent. Knowledge evolves; nothing is permanently forbidden.

---

3. System Overview

3.1 The Orchestrator

The orchestrator is a Python script that runs the generation loop. It holds no state in memory. It determines what to do by reading files:

- To know which generation it is running: it reads the `history/generations/` directory and counts existing snapshots.
- To know which agents to launch: it reads the Architect's manifest at `briefs/gen{NNN}/manifest.yaml`.
- To know if a phase completed: it checks for the expected output files from that phase.
- To know if the user intervened: it compares file modification timestamps against the last generation snapshot.

The orchestrator's job is mechanical:

1. Launch the Architect agent (always the same: point it to its prompt template and the project root).
2. Read the manifest the Architect wrote.
3. Launch the agents listed in the manifest, in parallel where specified, each pointed to its prompt template and brief.
4. After agents finish, launch debrief prompts for each.
5. Move agent outputs from workspaces to permanent locations.
6. Launch the Evaluator agent.
7. Move Evaluator outputs to knowledge directories.
8. Launch the System Critic agent.
9. Move System Critic outputs to feedback directories.
10. If this is a consistency review generation (or if the Evaluator flagged `strategic_shift: true`), launch the Consistency Reviewer.
11. Move Consistency Reviewer outputs.
12. Save the generation snapshot.
13. Check for target score. If met, stop. Otherwise, loop.

Every step is: read a file → launch a session → move the outputs. The orchestrator does not decide strategy. It does not pick agents. It does not evaluate solutions. It moves files.

Recovery. If the orchestrator crashes mid-generation, it restarts and inspects the file system. Which files exist tells it exactly where it stopped. Briefs exist but no agent outputs? Resume at step 3. Agent outputs exist but no evaluator report? Resume at step 6. This is possible because all state is in files.

3.2 The Generation Loop

Each generation proceeds through these phases:

Phase 1 — Planning. The orchestrator launches the Architect. The Architect reads the State of Affairs (Layer 0), all topic cluster summaries (Layer 1), the population summary, score history, agent reports from the previous generation, system feedback, and the latest consistency review. It assesses the strategic situation and writes a manifest plus a brief for each agent instance.

The manifest is a YAML file that the orchestrator reads to know what to launch. It lists every agent instance for this generation: type, instance number, model, and the path to its brief. It also lists any Experimentator tasks. The orchestrator does not interpret the manifest's strategic reasoning — it just launches what the manifest says to launch.

Phase 2 — Parallel work sessions. The orchestrator reads the manifest and launches all listed agent instances in parallel as Claude Code sessions. Each agent is told: "You are a {type} agent. Your brief is at {path}. Read your brief first. Write all output to your output/ directory." Each works autonomously: reading files, writing solutions, running the evaluation script, seeing scores, iterating, and submitting its best work. Experimentator instances run their experiments in their sandboxed directories.

Phase 3 — Debrief. The orchestrator sends each agent instance a follow-up prompt. Responses are written to agent output directories. The orchestrator moves them to `reports/{generation}/`.

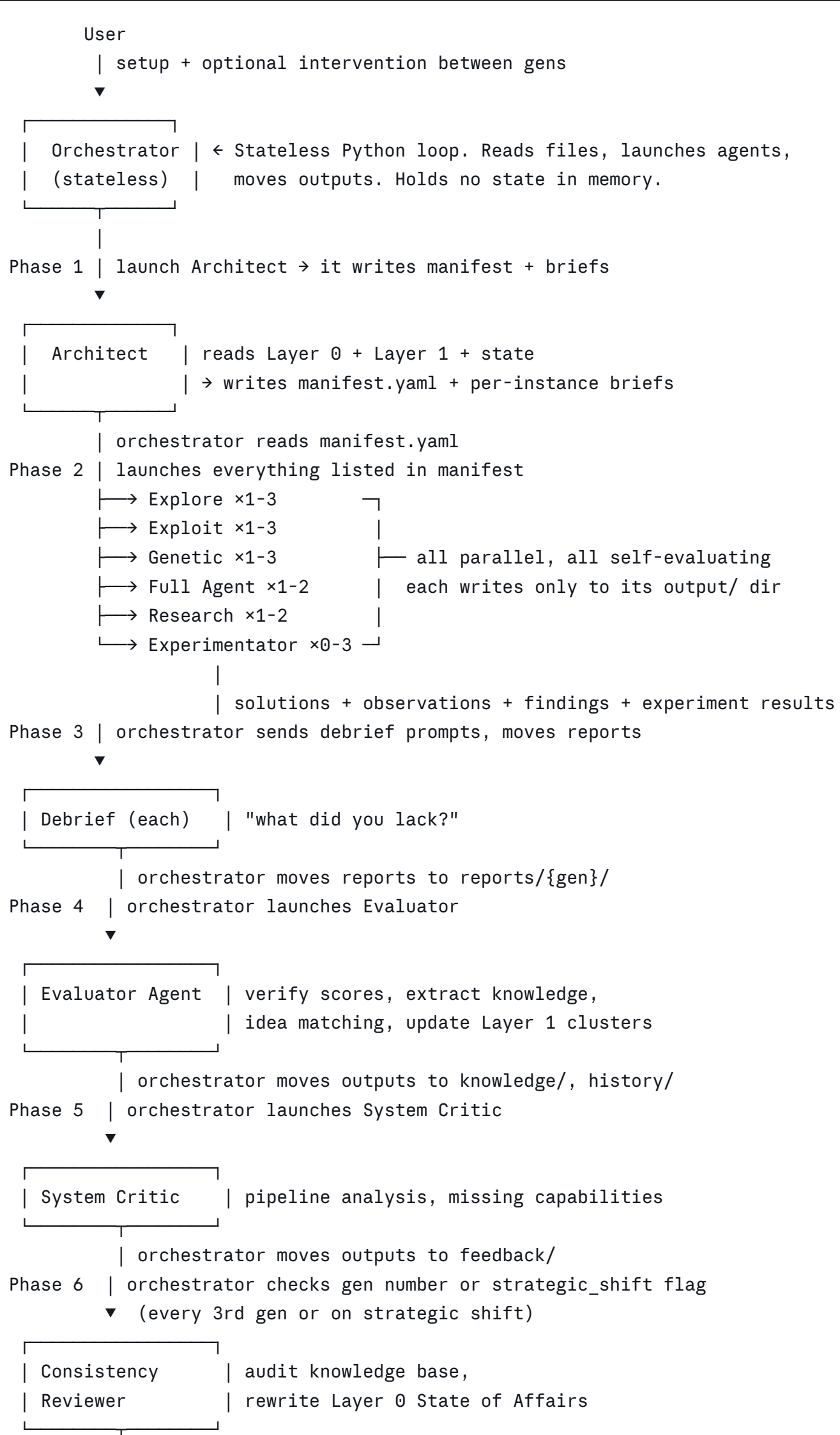
Phase 4 — Evaluation and knowledge update. The orchestrator launches the Evaluator agent, pointed to its prompt template and the current generation's outputs. The Evaluator reads all submitted solutions, re-runs the evaluation script to verify scores, extracts new knowledge, performs idea matching, updates the solution-idea map, and updates cluster summaries. The Evaluator writes all output to its output directory. The orchestrator moves the outputs to their permanent locations in `knowledge/`, `history/`, and `population/`.

Phase 5 — System critique. The orchestrator launches the System Critic, pointed to its prompt template. The System Critic reads all agent reports, observations, and feedback, and writes its analysis. The orchestrator moves outputs to `feedback/`.

Phase 6 — Consistency review (every 3 generations). The orchestrator checks the generation number. If it is a consistency review generation (or if the Evaluator's report contains `strategic_shift: true`), the orchestrator launches the Consistency Reviewer. The Reviewer audits the knowledge base, corrects clusters, and rewrites the State of Affairs. The orchestrator moves outputs to their permanent locations.

Phase 7 — State update. The orchestrator updates ranking symlinks (best.py, top/), saves a generation snapshot to `history/generations/`, and logs any detected user interventions.

3.3 The Architecture


User
setup + optional intervention between gens
Orchestrator
(stateless)
Orchestrator ← Stateless Python loop. Reads files, launches agents,
moves outputs. Holds no state in memory.
Phase 1 | launch Architect → it writes manifest + briefs
Architect
reads Layer 0 + Layer 1 + state
→ writes manifest.yaml + per-instance briefs
orchestrator reads manifest.yaml
Phase 2 | launches everything listed in manifest
Explore ×1-3
Exploit ×1-3
Genetic ×1-3
Full Agent ×1-2
Research ×1-2
Experimentator ×0-3
all parallel, all self-evaluating
each writes only to its output/ dir
solutions + observations + findings + experiment results
Phase 3 | orchestrator sends debrief prompts, moves reports
Debrief (each)
"what did you lack?"
orchestrator moves reports to reports/{gen}/
Phase 4 | orchestrator launches Evaluator
Evaluator Agent
verify scores, extract knowledge,
idea matching, update Layer 1 clusters
orchestrator moves outputs to knowledge/, history/
Phase 5 | orchestrator launches System Critic
System Critic
pipeline analysis, missing capabilities
orchestrator moves outputs to feedback/
Phase 6 | orchestrator checks gen number or strategic_shift flag
(every 3rd gen or on strategic shift)
Consistency
Reviewer
audit knowledge base,
rewrite Layer 0 State of Affairs

```
          |
          | orchestrator moves outputs to knowledge/, feedback/
Phase 7   ▼
     Orchestrator: update symlinks, save snapshot, check target
```

3.4 The Manifest

The Architect's manifest is the bridge between strategic intelligence and mechanical execution. It is a YAML file that the orchestrator reads literally. The orchestrator does not interpret strategy — it launches what the manifest says.

Example manifest:

yaml

```yaml
# briefs/gen007/manifest.yaml
generation: 7
strategy_summary: >
  Score stagnating at 0.89. Launching 3 explores for diversity
  in untried directions. 1 experimentator to test whether the
  score ceiling is metric-dependent. 1 exploit on best solution.
  1 genetic crossing memoization with greedy.

agents:
  - type: explore
    instance: 1
    model: sonnet
    brief: briefs/gen007/explore_1.md

  - type: explore
    instance: 2
    model: sonnet
    brief: briefs/gen007/explore_2.md

  - type: explore
    instance: 3
    model: sonnet
    brief: briefs/gen007/explore_3.md

  - type: exploit
    instance: 1
    model: sonnet
    brief: briefs/gen007/exploit_1.md

  - type: genetic
    instance: 1
    model: sonnet
    brief: briefs/gen007/genetic_1.md

  - type: experimentator
    instance: 1
    model: sonnet
    brief: briefs/gen007/experimentator_1.md

  - type: full
    instance: 1
    model: sonnet
    brief: briefs/gen007/full_1.md

  - type: research
    instance: 1
    model: sonnet
```

```
    brief: briefs/gen007/research_1.md

parallel_groups:
  - [explore_1, explore_2, explore_3, exploit_1, genetic_1, experimentator_1, full_1
```

The `strategy_summary` is for humans and analysis agents. The orchestrator ignores it. The orchestrator reads the `agents` list, creates a workspace for each, copies the brief, and launches Claude Code sessions. The `parallel_groups` field tells the orchestrator which agents can run simultaneously.

---

4. Agent Roster

Every agent is launched the same way by the orchestrator: a Claude Code session that receives its prompt template (from `agents/`) and the path to its brief (from `briefs/`). The prompt template defines the agent's role, rules, and output format. The brief defines this specific instance's task for this generation.

Every agent can read the entire project file system. Every agent writes only to its `workspace/{gen}_{type}_{instance}/output/` directory. The orchestrator moves outputs to permanent locations after the session ends.

4.1 Architect — The Coordinator

The Architect reads the State of Affairs (Layer 0), all topic cluster summaries (Layer 1), the population summary, and relevant Layer 2 files when needed. It writes a manifest (the execution plan for this generation) and a brief for each agent instance.

The manifest tells the orchestrator exactly what to launch. The briefs tell each agent what to focus on. The Architect is the only agent that decides the composition of each generation — how many of each agent type, what each instance should do, and whether to request Experimentator tasks.

The Architect decides how many instances of each agent type to launch (0 to max per type). If diversity is low, it launches 3 Explore instances with different directives. If the best solution is close to target, it launches 3 Exploit instances refining from different angles. If two promising but incompatible approaches exist, it launches 2 Genetic instances with different parent pairs. If there are open questions that experiments could answer, it launches 1-3 Experimentator instances with specific experiment briefs.

The Architect uses the coverage map section of the State of Affairs to ensure agents are directed toward unexplored territory. If the coverage map says "Memoization + heap has been explored thoroughly (ceiling 0.92). Memoization + DP has not been tried," the Architect directs an Explore instance toward memoization + DP and avoids assigning more work to memoization + heap.

Brief format example:

```
## Directive
Score stagnating at 0.89 for 2 generations. All top solutions
use memoization. You are explore instance 2 of 2 — instance 1
is trying heap approaches. You try non-comparison sorts.

## Read first
1. problem/description.md — the problem
2. knowledge/state_of_affairs.md — full system orientation
3. knowledge/clusters/caching_strategies.md — relevant cluster
4. population/best.py — format reference + what to beat
5. knowledge/facts/ — all files
```

Prompt template: `agents/architect.md` **Reads:** Layer 0 always. Layer 1 always. Layer 2 and other files as needed. **Writes to output/:** Manifest (`manifest.yaml`) + one brief per agent instance + manifest reasoning (`manifest_reasoning.md`). **Orchestrator moves to:** `briefs/{generation}/`

4.2 Explore — Divergent Search

The Explore agent finds fundamentally different approaches that the population has not tried. Its value comes from trying things other agents would not consider. It runs as a Claude Code work session: writes solutions, runs evaluate.py, iterates as many times as it decides, and submits its best work.

Multiple instances can run in parallel with different directives. Instance 1 might be told to try heap-based approaches; instance 2, non-comparison sorts; instance 3, hybrid methods. Each instance can produce multiple solutions.

The Explore agent reads the State of Affairs (Layer 0) first to understand what exists and what has been tried. It reads the topic cluster summaries (Layer 1) relevant to its directive to understand which specific sub-approaches have been explored within its assigned direction. It drills into Layer 2 files only when it needs to understand a specific idea or pattern in detail.

Core prompt essence: "Find something fundamentally different from what exists. Read the State of Affairs first — it tells you what's been tried and what hasn't. Write solutions, run evaluate.py to test them, iterate until satisfied. Even failed attempts are valuable — write observations about what you tried and what you learned."

Prompt template: `agents/explore.md` **Reads:** Layer 0 always. Layer 1 clusters as guided by brief. Layer 2 and other files as needed. **Writes to output/:** Solutions (`sol*.py`), observations (`observations.md`). **Orchestrator moves to:** `population/{generation}/explore_{instance}/`

4.3 Exploit — Depth-First Refinement

The Exploit agent takes a specific solution and makes it better. Micro-optimizations, edge case handling, parameter tuning, structural tightening. It sees the full code of its target solution and detailed knowledge about what makes it work.

Multiple instances can run in parallel, each refining a different solution. Instance 1 refines the best solution; instance 2 refines the second-best; instance 3 refines a promising hybrid.

The Exploit agent reads the State of Affairs (Layer 0) for orientation and the specific topic cluster summaries (Layer 1) relevant to its target solution. It drills into the individual idea and pattern files (Layer 2) that describe what makes the target solution work and what has been tried when refining similar solutions before.

Core prompt essence: "Here is a solution to refine. Make it better. Run evaluate.py after each change. Every change must have a rationale. If you hit a wall and can't improve further, say so — that's valuable information."

Prompt template: `agents/exploit.md` **Reads:** Layer 0 always. Layer 1 clusters relevant to target solution. Layer 2 ideas and patterns for the target. Full code of target solution. **Writes to output/:** Refined solutions (`sol*.py`), observations (`observations.md`). **Orchestrator moves to:** `population/{generation}/exploit_{instance}/`

4.4 Genetic — Crossover Synthesis

The Genetic agent combines the best parts of exactly 2 parent solutions into something better than either. The Architect selects parents for complementary strengths — not the top-2 by score, but two that use different approaches. The solution-idea map's "unexplored combinations" section guides parent selection.

Multiple instances can run in parallel with different parent pairs. Instance 1 might cross the memoization leader with the greedy leader; instance 2 might cross the heap approach with the hybrid approach.

The Genetic agent reads the State of Affairs (Layer 0) and the topic cluster summaries (Layer 1) for the clusters that contain the parents' ideas. It drills into the specific Layer 2 idea files that describe each parent's core approach and any known synergies or conflicts between them.

Core prompt essence: "Study these 2 parent solutions. Understand what each does well. Find a way to combine both strengths. The parents were chosen because they're different — the value is in the combination."

Prompt template: `agents/genetic.md` **Reads:** Layer 0 always. Layer 1 clusters relevant to both parents. Layer 2 ideas for each parent. Full code of both parent solutions. **Writes to output/:** Synthesized solutions (`sol*.py`), observations (`observations.md`). **Orchestrator moves to:** `population/{generation}/genetic_{instance}/`

4.5 Full Agent — Autonomous Problem Solver

The Full Agent is a skilled developer given a problem to solve with no restrictions. It gets the problem, the evaluation script, and full read access to every file in the project — all solutions, all knowledge, all history, all reports, all feedback. No one tells it what to do.

It might study the top solutions and synthesize something better. It might ignore everything and build from scratch. It might read the knowledge base, find a debunked idea that was debunked for the wrong reasons, and resurrect it. It might read agent reports, spot a pattern no one noticed, and exploit it.

The Full Agent exists because the best solution might come from ignoring all the structure the system has built. While other agents operate within strategic roles defined by the Architect, the Full Agent bypasses all curation and just works on the problem directly.

The Full Agent reads the State of Affairs (Layer 0) for orientation but is free to dive into any level of the hierarchy or ignore it entirely. It has no prescribed reading pattern.

The Full Agent can also write experiment requests to `output/experiment_requests.md`. These are questions or tests it wants the Experimentator to run in a future generation. The Architect reads these requests when planning the next generation.

Core prompt essence: "You are a skilled developer. Here is a problem and an evaluation script. All project files are available to read. The State of Affairs gives you a quick overview. Solve it however you want. No restrictions. If you want a controlled experiment tested, write it to experiment_requests.md."

Prompt template: `agents/full.md` **Reads:** Everything. No brief needed — or a one-line brief at most. **Writes to output/:** Solutions (`sol*.py`), observations (`observations.md`), optionally experiment requests (`experiment_requests.md`). **Orchestrator moves to:** `population/{generation}/full_{instance}/`

4.6 Experimentator — Controlled Knowledge Producer

The Experimentator does not try to solve the problem. It runs controlled experiments that answer specific questions, producing knowledge rather than solutions. Every other solution-producing agent is incentivized to maximize score. The Experimentator is incentivized to maximize information.

The Architect creates Experimentator tasks when there are open questions that could be resolved by targeted testing. The Full Agent can also request experiments. Typical experiment briefs:

- "Does sorting the input improve score for inputs above 10000 elements? Test with and without sorting on large test cases."
- "Is the 0.92 score ceiling a property of memoization approaches or of the evaluation metric? Run the best solution with artificially simple inputs to check."
- "What happens if we remove the cache entirely from the top solution? How much does the score drop, and on which test cases?"
- "Test three different hash functions for the lookup table. Which gives the best cache hit rate?"

The Experimentator receives a brief with a specific question, a methodology suggestion, and relevant file references. It designs the experiment, writes test scripts, runs them, and reports results. Its output is structured findings — not solutions.

The Experimentator runs all code inside a `sandbox/` subdirectory within its workspace. This isolates its test scripts, temporary files, and experiment artifacts from the main project. The Experimentator can copy files it needs (solutions, evaluate.py) into its sandbox but runs everything there.

Core prompt essence: "You are running a controlled experiment. Your question is stated in your brief. Design a fair test, run it, and report what you found. Be precise about methodology and results. Your job is to produce reliable knowledge, not good scores. Write all test code in your sandbox/ directory."

Prompt template: `agents/experimentator.md` **Reads:** Layer 0, Layer 1 clusters relevant to the experiment, Layer 2 files as needed, specific solutions referenced in brief. **Writes to output/:** Experiment results (`experiment_results.md`), raw data (`sandbox/` directory contents preserved for audit). Does NOT write solutions to the population. **Orchestrator moves to:** `knowledge/experiments/{generation}/experimentator_{instance}/`

4.7 Research — Knowledge Gatherer

The Research agent investigates techniques, algorithms, and approaches relevant to the problem. It does not produce solutions — it produces knowledge that helps other agents write better solutions.

The Architect suggests research questions in the brief, but the Research agent has full autonomy to pursue its own direction. The Architect might not know the right questions. Agent gap reports — where agents wrote "I needed to know X" — are a primary input for Research.

The Research agent reads the State of Affairs (Layer 0) and all topic cluster summaries (Layer 1) to understand what the system already knows. It drills into Layer 2 files in areas where the cluster summaries indicate gaps, disputed claims, or thin evidence.

Core prompt essence: "Find knowledge that helps solve this problem. The Architect suggests questions, but follow your own judgment if you see a better angle. Read the State of Affairs and cluster summaries to understand what we already know. Check agent gap reports for what others needed. Be specific about how findings apply to our problem."

Prompt template: `agents/research.md` **Reads:** Layer 0 always. All Layer 1 clusters. Layer 2 as needed. Agent gap reports. **Writes to output/:** Findings (`findings.md`). **Orchestrator moves to:** `knowledge/research/{generation}/research_{instance}/`

4.8 Evaluator Agent — Knowledge Extraction, Score Verification, and Layer 1 Maintenance

The Evaluator agent runs after all work sessions complete. It is the primary knowledge worker: it verifies scores, extracts ideas and patterns from results, manages the knowledge lifecycle, maintains the solution-idea map, and updates the topic cluster summaries (Layer 1) to reflect what was learned this generation.

First, it re-runs evaluate.py on every submitted solution. Agents self-evaluate during their sessions, but they might have bugs in how they call the script, or might misreport scores. Trust but verify. Score discrepancies are themselves information.

Then it analyzes all results, observations, and experiment results:

- Creates new ideas, patterns, and facts from what agents discovered (Layer 2)
- Updates existing knowledge with new evidence (supporting, contradicting, or refining)
- Manages lifecycle transitions (active → established, active → disputed, etc.)
- Does idea matching: which ideas does each solution implement, as central or peripheral
- Updates the solution-idea map with new entries and recomputes aggregated stats
- Generates the coverage matrix appendix from the solution-idea map (structured data showing which idea combinations have been tried)
- Incorporates Experimentator results as high-confidence evidence (controlled experiments produce stronger evidence than incidental observations)
- Performs qualitative assessment: is this code elegant, generalizable, fragile?

After updating individual knowledge files, the Evaluator updates the topic cluster summaries (Layer 1):

- Updates the cluster summaries that were affected by this generation's results
- Creates new clusters when a new idea does not fit any existing cluster
- Merges clusters when two clusters have converged (their ideas now overlap significantly)
- Updates each cluster's aggregated statistics (best score, idea count, lifecycle distribution)
- Flags clusters where contradictory evidence has emerged

The Evaluator does not rewrite the State of Affairs (Layer 0) — that is the Consistency Reviewer's responsibility on its periodic schedule. The Evaluator focuses on keeping Layer 1 current, Layer 2 accurate, and the coverage matrix up to date. If this generation produced a strategic shift (new best score breaking long stagnation, leading approach debunked, completely novel technique), the Evaluator flags `strategic_shift: true` in its report.

Core prompt essence: "Verify all scores. Analyze what happened — including experiment results. Extract knowledge. Update the cluster summaries for anything that changed. Update the coverage matrix. Every claim must have evidence. Be precise about causation vs correlation. Ideas are strategies, patterns are observations, facts are truths."

Prompt template: `agents/evaluator.md` **Reads:** Everything, with focus on this generation's submitted solutions, observations, experiment results, and the existing Layer 1 clusters that relate to this generation's work. **Writes to output/:** New/updated knowledge files (Layer 2), updated cluster summaries (Layer 1), updated `solution_idea_map.md`, updated `coverage_matrix.md`, generation snapshot, evaluator report. **Orchestrator moves to:** `knowledge/` (ideas, patterns, facts, clusters), `history/` (solution-idea map, coverage matrix, snapshot).

4.9 System Critic — Pipeline Analyst

The System Critic runs after the Evaluator. It looks at the system itself, not the solutions. It reads all agent debriefs, observations, and feedback, and identifies:

- **Pipeline problems:** Evaluation takes too long. Agents are context-starved. Briefs are too vague or too prescriptive.
- **Missing capabilities:** Research agent can't access scientific papers. No web search available. Evaluation script doesn't report per-test-case breakdowns.
- **Prompt problems:** Explore keeps producing similar solutions despite different directives. Genetic can't effectively cross solutions that use incompatible paradigms.
- **Resource issues:** Agents report inability to download files, missing libraries, environment limitations.
- **Knowledge quality issues:** Cluster summaries are getting stale. State of Affairs is missing important context. Agents report that the knowledge hierarchy is not reflecting reality.
- **Experiment gaps:** Open questions that the Experimentator should test. The System Critic can recommend experiment tasks for the Architect to consider.
- **Actionable recommendations:** What the user should add, change, or configure. Written as a prioritized list with reasoning.

The System Critic is the agent that makes the system self-aware about its own limitations. Its output is the primary thing the user reads between generations.

Core prompt essence: "Read all agent reports and feedback. What is broken in the pipeline? What capabilities are missing? Are agents getting the right knowledge at the right level of detail? What experiments would answer open questions? What should the user change? Write actionable recommendations."

Prompt template: `agents/system_critic.md` **Reads:** Everything, with focus on `reports/`, `feedback/`, and `knowledge/observations/`. **Writes to output/:** System analysis (`system_analysis.md`), updated system recommendations (`system_recommendations.md`), experiment suggestions (`experiment_suggestions.md`). **Orchestrator moves to:** `feedback/system_analysis/`, `feedback/system_recommendations.md`, `feedback/experiment_suggestions/`.

4.10 Consistency Reviewer — Knowledge Auditor and Layer 0 Maintainer

The Consistency Reviewer runs every 3 generations. The Evaluator processes what is new each generation and keeps Layer 1 current. The Consistency Reviewer checks what is old — it audits the entire knowledge base against current evidence, and then rewrites the State of Affairs document (Layer 0) from scratch.

Knowledge audit. For each piece of knowledge (Layer 2), the Consistency Reviewer either confirms (updates `last_confirmed_gen`), recommends refinement (scope needs narrowing), disputes (contradictory evidence found), debunks (proven wrong with explanation), or flags as stale (not confirmed in 5+ generations).

Its unique value is cross-consistency analysis: finding contradictions between separate ideas, identifying unexplored connections between knowledge pieces, and spotting gaps that no single-generation analysis would catch.

Agent-reported doubts ("I think X might be wrong" from debrief reports) are the Reviewer's highest-priority investigation targets — agents are closest to the actual problem-solving.

Cluster review. After auditing individual knowledge, the Consistency Reviewer reviews the topic cluster summaries (Layer 1) for accuracy. It verifies that cluster summaries accurately reflect the Layer 2 files they contain, that no ideas have been misclustered, and that cluster-level statistics and status are correct. If the Evaluator's incremental updates have introduced drift, the Consistency Reviewer corrects it.

State of Affairs rewrite. After completing the audit, the Consistency Reviewer rewrites the State of Affairs document (Layer 0) from scratch. This document is the system's compressed self-knowledge. It reads the structured coverage matrix (maintained by the Evaluator) alongside the full knowledge base to produce an accurate narrative. The State of Affairs includes:

- **Current standing:** Best score, how many generations have run, overall trajectory (improving, stagnating, declining)
- **What works:** The 2-4 main approaches that produce the best results, described at a high level
- **Current frontier:** Why the system is not improving further, what the blocking problem appears to be
- **Coverage map:** What idea directions have been explored (with depth: lightly tried, thoroughly explored, exhausted), what idea directions have not been tried, what combinations have and have not been attempted, and which untried directions look most promising based on available evidence. The coverage map is written as a narrative but cross-checked against the structured coverage matrix for accuracy.
- **Dead ends:** Approaches that have been tried and proven unproductive, with brief reasons why
- **Open questions:** Unresolved disputes, thin-evidence claims, agent-reported doubts, and experiments that should be run

The State of Affairs is kept short — 800-1500 tokens. It is a narrative, not a data dump. Its purpose is to let any agent reading it for the first time understand the full strategic situation in under a minute.

Core prompt essence: "Audit every idea, pattern, and fact. Is it still true? What contradicts it? What connects to it that nobody noticed? Agent doubts are your top priority. After the audit, rewrite the State of Affairs from scratch — use the coverage matrix to ensure accuracy. Make it complete and short."

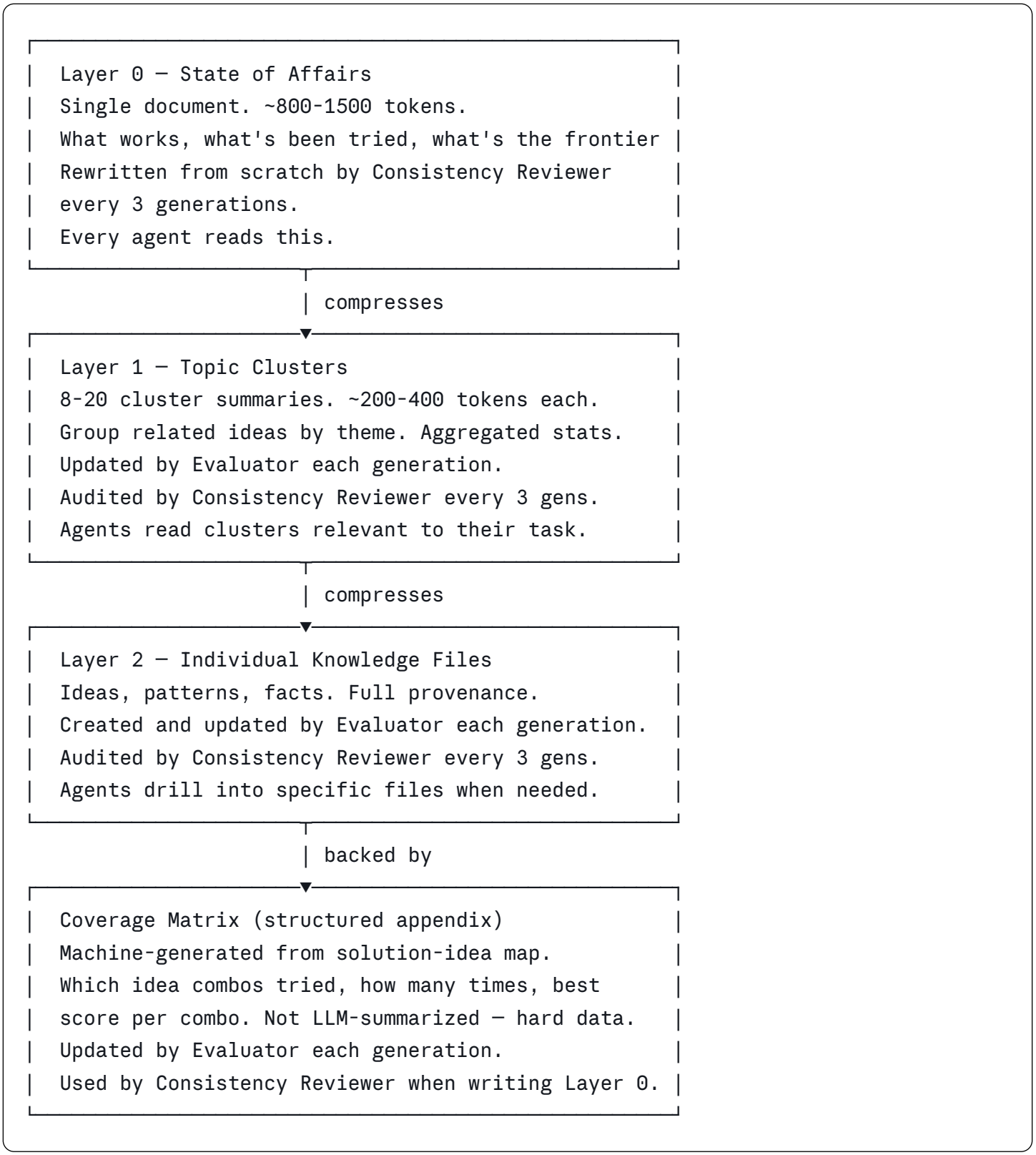
Prompt template: `agents/consistency_review.md` **Reads:** Everything. Full knowledge base at all layers, coverage matrix, recent results, all agent reports. **Writes to output/:** Consistency review report, updated knowledge files (Layer 2), corrected cluster summaries (Layer 1), rewritten State of Affairs (Layer 0). **Orchestrator moves to:** `feedback/consistency_reviews/`, `knowledge/` (all layers).

---

5. Knowledge Model

5.1 Three Layers, One System

Knowledge is organized as a three-layer hierarchy designed to scale across many generations without overwhelming agent context windows. Each layer compresses the one below it. Agents read top-down: Layer 0 first for orientation, Layer 1 for relevant themes, Layer 2 only when they need full detail on a specific piece of knowledge.



All three layers live in the `knowledge/` directory. Layer 0 is a single file. Layer 1 is a directory of cluster summaries. Layer 2 is the set of individual idea, pattern, and fact files. The coverage matrix lives in `history/coverage_matrix.md` alongside the solution-idea map.

5.2 Layer 0 — State of Affairs

The State of Affairs is a single markdown document that answers the question: "If you know nothing about this project, what do you need to know to contribute effectively right now?"

It is written as a narrative, not a list of facts. It covers:

- **Current standing:** Best score, generation count, overall trajectory
- **What works:** The main approaches producing the best results
- **Current frontier:** Why the system is stuck or where it is pushing
- **Coverage map:** What has been explored (lightly, thoroughly, exhausted), what has not been tried, what combinations have been attempted, which untried directions look most promising
- **Dead ends:** Approaches that failed and why
- **Open questions:** Unresolved disputes, thin claims, agent doubts, recommended experiments

The State of Affairs is rewritten from scratch by the Consistency Reviewer every 3 generations. Between consistency reviews, it may be up to 3 generations stale, but this is acceptable because it captures strategic-level knowledge that changes slowly. The coverage map section is cross-checked against the structured coverage matrix so it reflects hard data, not just LLM summarization.

Maintained by: Consistency Reviewer (full rewrite every 3 generations). **Read by:** Every agent, every generation. This is always the first knowledge file an agent reads. **Size target:** 800-1500 tokens. If it grows beyond this, the Consistency Reviewer is instructed to compress harder.

Example (generation 12):

markdown

```
---
generation: 12
best_score: 0.92
trajectory: stagnating
last_updated_gen: 12
---

# State of Affairs — Generation 12

## Current Standing

Best score is 0.92, achieved in generation 5 and matched but not
beaten in generations 7 and 9. We have run 12 generations with
74 total solutions evaluated. The system is stagnating — no
improvement in 7 generations.

## What Works

Two approach families dominate. Memoization-based solutions
(cluster: caching_strategies) consistently score 0.85-0.92,
with the ceiling appearing to be cache thrashing on large inputs.
Greedy with pruning (cluster: greedy_approaches) scores
0.78-0.88, with strength on large inputs where memoization
struggles.

## Current Frontier

The blocking problem is large-input performance. Memoization
hits a ceiling at ~10000 elements due to memory pressure.
Greedy handles large inputs well but lacks the precision of
memoization on small inputs. No solution has successfully
combined both strengths — the 3 attempts at crossing them
(gen 6, 8, 10) all degraded to one parent's approach.

## Coverage Map

**Thoroughly explored:**
- Memoization with various cache strategies (LRU, bounded,
  adaptive) — 12 solutions, ceiling 0.92
- Greedy with pruning variants — 8 solutions, ceiling 0.88
- Memoization × greedy crossover — 3 attempts, all failed
  to integrate meaningfully

**Lightly explored:**
- Heap-based approaches — 2 solutions (gen 4), best 0.71,
  abandoned early, may deserve revisiting
- Divide-and-conquer — 1 solution (gen 3), scored 0.65,
```

```
  implementation was naive

**Not yet explored:**
- Non-comparison-based approaches (radix, counting, bucket)
- Adaptive algorithms that switch strategy based on input size
- Preprocessing/transformation before main algorithm
- Parallel or segmented approaches

**Most promising untried direction:** Adaptive strategy
switching — use memoization for small segments, greedy for
large ones. Both approaches work well individually in their
strength domain.

## Dead Ends

- Unbounded memoization: cache thrashing above 10000 elements.
  Always use bounded cache. (idea_042, debunked as universal
  strategy in gen 7)
- Pure recursion: hits recursion limit at depth 1000.
  (fact_003)
- Sorting the full input before processing: O(n log n)
  overhead not recovered by simpler inner loop. (idea_019,
  debunked gen 5)

## Open Questions

- Is the 0.92 ceiling a property of memoization approaches
  or of the evaluation metric? (raised by exploit_1 in gen 9;
  experimentator task recommended)
- Could heap approaches work if combined with memoization
  rather than used standalone? (never tested)
- Agent gap: multiple agents have requested understanding of
  the score function's weighting across test cases
```

5.3 Layer 1 — Topic Clusters

Topic clusters group related ideas into themes. Each cluster is a single markdown file with YAML frontmatter, summarizing a family of related ideas, their collective evidence, and their current status.

A cluster is not a category imposed from the outside — it emerges from the ideas themselves. When the Evaluator creates a new idea that is closely related to existing ideas, it adds the idea to an existing cluster. When a new idea does not fit any cluster, the Evaluator creates a new one. When two clusters converge, the Evaluator merges them.

Each cluster summary contains:

- The cluster's thesis: what this family of ideas is about
- The ideas it contains (by ID, with lifecycle status and top score)
- Aggregated statistics: best score across all ideas in the cluster, average, how many solutions use ideas from this cluster
- Current status: is this cluster actively productive, stagnating, or exhausted?
- Known limitations: what the cluster's approaches cannot do
- Synergies and conflicts: which other clusters' ideas work well or poorly with this cluster's ideas
- A "for agents" section: practical guidance for anyone working in this area

Clusters are kept short — 200-400 tokens each. They are summaries, not inventories.

Maintained by: Evaluator (incremental updates each generation). Consistency Reviewer (accuracy audit every 3 generations). **Read by:** Agents read clusters relevant to their task as directed by the Architect's brief. The Architect reads all clusters. **Size target:** 8-20 clusters active at any time. If the count exceeds 20, the Evaluator merges the most closely related clusters.

Example file:

markdown

```
---
id: cluster_caching
type: cluster
theme: "Caching and memoization strategies"
idea_count: 5
active_ideas: 3
established_ideas: 1
debunked_ideas: 1
best_score: 0.92
avg_score: 0.79
solutions_using: 18
status: stagnating
last_updated_gen: 11
ideas: [idea_042, idea_038, idea_055, idea_061, idea_019]
synergies: [cluster_preprocessing]
conflicts: [cluster_greedy]
---

# Caching and Memoization Strategies

Approaches that cache intermediate results to avoid redundant
computation. This is the highest-scoring cluster (0.92) but
has stagnated — no improvement since generation 5.

## What works

Bounded LRU memoization on the inner loop (idea_042,
established, top score 0.92). Cache size should be proportional
to K. Adaptive cache sizing (idea_055, active, top score 0.90)
shows promise but needs more testing.

## What doesn't work

Unbounded memoization (idea_019, debunked) — cache thrashing
on inputs above 10000 elements. Full-result caching (idea_061,
active, top score 0.72) — overhead exceeds benefit for most
input sizes.

## Current ceiling and why

The 0.92 ceiling appears to be caused by memory pressure on
large inputs. All memoization solutions degrade above ~10000
elements. The open question is whether a hybrid approach
(memoization for small segments, different strategy for large
ones) could break through.

## For agents
```

```
If you work with memoization: always use bounded cache with
LRU eviction, size proportional to K. If you're trying to
break 0.92, the path is probably not better caching — it's
combining caching with a different strategy for the large-input
regime. See cluster_greedy for the complementary approach.
```

5.4 Layer 2 — Individual Knowledge Files

5.4.1 Ideas — Implementation Strategies

Ideas are claims about how to solve the problem: "use memoization on the inner loop," "pre-sort the input," "use a min-heap of size K." They are strategies that can be combined, that have synergies and conflicts, and that can be central or peripheral to a solution.

Lifecycle: active → established → disputed → debunked → archived

- **Active:** Fresh or recently updated. Some evidence, not yet confirmed across multiple generations.
- **Established:** Confirmed by multiple agents or generations. Reliable.
- **Disputed:** Both supporting and contradicting evidence exist. Needs investigation.
- **Debunked:** Proven wrong. Kept with explanation as a warning. Can be revived if new evidence contradicts the debunking.
- **Archived:** Not wrong, just old and irrelevant. Not shown to agents.

Ideas participate in a many-to-many relationship with solutions. Each link records whether the idea was central or peripheral to that solution. This enables aggregated statistics: how many solutions use this idea, what are the average, median, and top scores, which combinations of ideas have been tried.

Every idea belongs to exactly one topic cluster. The cluster ID is recorded in the idea's frontmatter. When a new idea is created, the Evaluator assigns it to a cluster or creates a new one.

Size limits: As the idea pool grows, the bar for adding new ones rises. Under 30 ideas: must be good. 30-50: must be very good. 50-100: must be revolutionary. Over 100: merge or cluster instead. This prevents context bloat organically.

Example file:

markdown

```
---
id: idea_042
type: idea
cluster: cluster_caching
lifecycle: established
certainty: high
created_gen: 3
created_by: evaluator
last_confirmed_gen: 9
last_confirmed_by: consistency_review
tags: [memoization, inner-loop, caching]
solutions:
  - {id: gen005_exploit_1_sol02, score: 0.92, role: central}
  - {id: gen005_exploit_1_sol01, score: 0.89, role: central}
  - {id: gen007_explore_1_sol01, score: 0.58, role: central}
stats:
  times_used: 3
  avg_score: 0.76
  top_score: 0.92
  median_score: 0.89
related: [idea_038, pattern_003]
---

Memoization on the inner loop consistently improves performance
for inputs under 10000 elements.

## Evidence

Gen 3 (evaluator): Solution gen003_explore_1_sol01 used memoization
and scored 0.92. Solution gen003_explore_1_sol02 without it scored
0.65. Same approach otherwise — clear causal link.

Gen 5 (evaluator): Exploit applied memoization to a greedy solution.
Score jumped from 0.71 to 0.89. Independent confirmation.

Gen 7 (evaluator): Solution gen007_explore_1_sol01 used memoization
but scored only 0.58. Cache thrashing on inputs > 10000.

Gen 9 (consistency_review): Confirmed. Still in top-3 solutions.

Gen 11 (experimentator_1): Controlled test — removed cache from
top solution, score dropped from 0.92 to 0.64. Cache is the
primary contributor to score on small/medium inputs.

## Current understanding

Works well with bounded or LRU cache. Without bounds, degrades
```

```
on large inputs due to memory pressure.

## For agents

If you use memoization, add a cache size bound proportional to K.
```

5.4.2 Patterns — Empirical Observations

Patterns are discovered conditions and correlations: "solutions score higher when K < 10," "the score ceiling for memoization approaches is 0.92," "pre-sorting hurts heap approaches." They qualify ideas — "memoization helps" (idea) + "but only for K < 15" (pattern) = complete understanding.

Lifecycle: active → confirmed → outdated. Simpler than ideas because patterns are observations, not hypotheses.

Every pattern belongs to one or more topic clusters. Patterns that qualify ideas are assigned to the same cluster as those ideas. Patterns that span multiple clusters (cross-cutting observations) are assigned to all relevant clusters.

5.4.3 Facts — Environment Truths

Facts are true statements about the environment: "NumPy unavailable," "heapq standard library is available," "test cases have K from 2 to 50," "recursion limit is 1000." They have no lifecycle — they are either current or corrected.

All agents have access to all facts. Facts are cheap to include and expensive to miss. A fact marked `critical: true` is highlighted in briefs.

Facts do not belong to topic clusters. They are global context that applies across all approaches. They live in a flat directory and are included in every agent's reading list.

5.4.4 Observations — Unprocessed Raw Material

Observations are raw notes from agent sessions that have not been processed by the Evaluator yet. They live in `knowledge/observations/` and are consumed exclusively by the Evaluator and System Critic during their analysis phases.

Observations do not participate in the hierarchy. They are input to the Evaluator, which processes them into Layer 2 knowledge and updates Layer 1 clusters accordingly.

5.4.5 Experiment Results

Experiment results from the Experimentator are a special category of evidence. Because they come from controlled tests with clear methodology, they carry higher evidential weight than incidental observations from solution-producing agents. The Evaluator treats experiment results as strong evidence when updating ideas, patterns, and clusters.

Experiment results live in `knowledge/experiments/{generation}/` and are preserved with their full methodology for auditability.

5.5 The Coverage Matrix

The coverage matrix is a structured appendix that records which idea combinations have been tried, how many times, and the best score for each combination. Unlike the coverage map in the State of Affairs (which is a narrative written by the Consistency Reviewer), the coverage matrix is hard data derived from the solution-idea map.

The Evaluator generates the coverage matrix each generation from the solution-idea map. It is a cross-tabulation: rows are ideas, columns are ideas, cells are `{count, best_score}` for the number of solutions that used both ideas and the best score achieved.

The coverage matrix prevents the most insidious failure mode of long-running evolutionary systems: the LLM-summarized coverage map in the State of Affairs gradually drifts from reality (because it is rewritten from memory every 3 generations), and the system starts re-exploring territory it has already covered. The coverage matrix is not LLM-summarized — it is computed from structured data. The Consistency Reviewer reads both the matrix and the full knowledge base when writing the coverage map section of the State of Affairs, ensuring the narrative is grounded in data.

Maintained by: Evaluator (generated from solution-idea map each generation). **Read by:** Consistency Reviewer (when writing Layer 0 coverage map), Architect (when planning Genetic crossovers and identifying untried combinations). **Location:** `history/coverage_matrix.md`

5.6 How the Layers Stay in Sync

The three layers are maintained by different agents at different frequencies:

Layer	Content	Maintained by	Frequency	Size target
0	State of Affairs	Consistency Reviewer	Every 3 generations (full rewrite)	800-1500 tokens
1	Topic Clusters	Evaluator (incremental), Consistency Reviewer (audit)	Every generation (Evaluator), every 3 (Reviewer)	200-400 tokens each, 8-20 clusters
2	Individual files	Evaluator (create/update), Consistency Reviewer (audit)	Every generation (Evaluator), every 3 (Reviewer)	No individual limit, managed by idea size limits
—	Coverage Matrix	Evaluator (regenerated)	Every generation	Grows with idea count, structured data

Layer drift. Because the State of Affairs (Layer 0) is only rewritten every 3 generations, it can become stale between consistency reviews. This is acceptable because Layer 0 captures strategic-level knowledge that changes slowly — the best approaches, the coverage map, the frontier problem. Tactical changes (a new idea, an updated pattern) are captured in Layer 1 by the Evaluator each generation. Agents always read both layers, so they get a stable strategic picture from Layer 0 and current tactical detail from Layer 1.

Emergency updates. If a generation produces a result that fundamentally changes the strategic picture — a new best score that breaks a long stagnation, a debunking of the leading approach, a completely novel technique — the Evaluator flags this in its generation report as `strategic_shift: true`. The orchestrator checks for this flag and triggers an early Consistency Review (and thus a Layer 0 rewrite) after such events, outside the normal 3-generation cycle. This is configurable.

---

6. The Debrief System

6.1 How It Works

After every agent instance completes its work session, the orchestrator sends it one follow-up prompt:

```
You just completed your task for generation N.
Here's what you produced: [summary of solutions + scores]

1. What information did you lack that would have helped?
2. What facts you were given do you think might be wrong or outdated?
3. Was the State of Affairs accurate? Was anything missing or misleading?
4. What would you do differently with more or different context?
5. Are there specific experiments that would answer questions you encountered?
```

The response is written to the agent's output directory. The orchestrator moves it to `reports/{generation}/{agent_type}_{instance}.md`.

6.2 Who Reads Debriefs

Architect: Reads all reports before writing briefs for the next generation. Adjusts reading lists and directives based on what agents said they lacked. Considers experiment requests from debriefs and from the System Critic when deciding whether to launch Experimentator instances.

Evaluator agent: Reads all reports looking for knowledge that needs correction ("I think idea X might be wrong"), signals about knowledge accuracy, and observations worth tracking. Also looks for feedback about cluster accuracy — agents may report that a cluster summary is misleading or missing important information.

System Critic: Reads all reports looking for pipeline problems ("evaluation is too slow"), missing capabilities ("I needed web access"), and prompt issues ("my directive was too vague"). Also looks for knowledge hierarchy problems — agents may report that the State of Affairs was stale, or that they couldn't find the right cluster for their task.

Research agent: Reads `feedback/agent_gaps.md` (synthesized by the Evaluator from debriefs) looking for knowledge gaps it could fill.

Consistency Reviewer: Reads all reports looking for "I think X might be wrong" statements. These become its highest-priority investigation targets. Also reads feedback about the State of Affairs accuracy to incorporate into the next rewrite.

User: Can read any report at any time for transparency into the system's operation.

6.3 The Evaluator and System Critic Get Debriefed Too

The Evaluator might report: "I couldn't tell if improvements came from algorithm changes or parameter tuning — the evaluation script should report more granular metrics."

The System Critic might report: "Three agents independently mentioned the same knowledge gap. I've written it as a recommendation but nothing has changed across 4 generations."

These meta-debriefs go to the user via `feedback/system_recommendations.md`.

---

7. Multi-Instance Agents

7.1 How It Works

The Architect decides how many instances of each agent type to launch per generation by writing them into the manifest. Minimum 0, maximum defined in config (typically 3 for solution agents, 2 for research and full, 3 for experimentator). The orchestrator reads the manifest and launches exactly what it says.

Each instance gets its own brief with a distinct directive. The Architect writes a manifest reasoning document explaining the strategic logic.

7.2 When to Use Multiple Instances

Explore ×3: When population diversity is low. Each instance gets a different search direction. The Architect uses the coverage map from the State of Affairs to assign each instance to a different unexplored direction.

Exploit ×3: When multiple viable solutions exist near the target score. Each instance refines a different solution.

Genetic ×2-3: When the solution-idea map shows multiple unexplored combinations worth trying.

Experimentator ×1-3: When there are multiple independent questions to test. Each instance gets a different experiment. The Architect draws experiment tasks from: its own strategic questions, Full Agent experiment requests, System Critic experiment suggestions, and open questions in the State of Affairs.

Full ×2: Rarely — the full agent is expensive. Used when the system suspects its structured approach might be limiting.

Research ×2: When there are knowledge gaps in multiple unrelated areas.

7.3 Instance Coordination

Instances of the same type do not communicate directly. They run in parallel without knowledge of each other. Coordination happens through the Architect: it writes different directives for each instance. Instance 1 of Explore might be told "try heap approaches" while instance 2 is told "try non-comparison sorts."

The manifest reasoning records the plan so that the Evaluator and System Critic can assess whether the parallelism was well-utilized.

---

8. Many-to-Many: Solutions and Ideas

8.1 Purpose

The solution-idea map tracks which ideas are implemented in which solutions and how central each idea is. This enables the Architect to identify unexplored combinations for the Genetic agent, the Evaluator to assess idea impact quantitatively, the Evaluator to generate the coverage matrix, the Consistency Reviewer to verify ideas against their aggregate track record, and the coverage map in the State of Affairs to accurately reflect what has been tried.

8.2 How It Is Built

When the Evaluator processes a generation's results, it performs idea matching: it reads each solution's code alongside the current ideas, and classifies which ideas the solution implements and whether each is central or peripheral. This classification is written to the solution file header and aggregated into `history/solution_idea_map.md`.

From the solution-idea map, the Evaluator generates the coverage matrix (`history/coverage_matrix.md`): a structured cross-tabulation of which idea pairs have been tried together, how many times, and the best score for each combination.

8.3 What It Contains

Per idea: number of solutions using it, average score, median score, top score, how often it appears as central vs peripheral.

Per solution: which ideas it implements, with roles.

Combination analysis: which idea pairs have been tried together, which have not, and which untried combinations look most promising based on individual idea performance.

The combination analysis section feeds directly into the coverage matrix (structured data) and the coverage map of the State of Affairs (narrative). Together they prevent the system from retreading explored ground even over very long runs.

---

9. File Structure

9.1 Naming Conventions

Agents are identified as `{type}_{instance}`: `explore_1`, `exploit_2`, `genetic_3`.

Solutions are identified as `gen{NNN}_{type}_{instance}_sol{NN}`: `gen007_explore_2_sol03`.

Knowledge is identified as `{type}_{sequential_id}`: `idea_042`, `pattern_003`, `fact_015`.

Clusters are identified as `cluster_{theme}`: `cluster_caching`, `cluster_greedy`, `cluster_heap`.

9.2 Directory Layout

```
alpha-evolve/
├── problem/                          # User-created problem definition (read-only
for agents)
│   ├── description.md
│   ├── constraints.md
│   ├── evaluate.py                   # Agents run this themselves
│   └── test_cases/
│
├── population/                       # All solutions (written only by
orchestrator)
│   ├── best.py                       # Symlink to highest-scoring solution
│   ├── top/                          # Top 10 by score (ranked symlinks)
│   ├── gen007/                       # Solutions from generation 7
│   │   ├── explore_1/                # 1st explore instance
│   │   │   ├── sol01.py
│   │   │   ├── sol02.py
│   │   │   └── sol03.py
│   │   ├── explore_2/                # 2nd explore instance
│   │   │   └── sol01.py
│   │   ├── exploit_1/
│   │   │   └── sol01.py
│   │   ├── genetic_1/
│   │   │   └── sol01.py
│   │   ├── full_1/
│   │   │   ├── sol01.py
│   │   │   └── sol02.py
│   │   └── experimentator_1/        # Experiments, not solutions
│   │       └── (no sol files — experiments don't enter population)
│   └── summary.md                    # Auto-generated approach categories + stats
│
├── knowledge/                        # Accumulated intelligence (3 layers)
│   │                                 # Written only by orchestrator from agent
outputs
│   ├── state_of_affairs.md           # Layer 0 — single strategic overview
│   ├── clusters/                     # Layer 1 — topic cluster summaries
│   │   ├── cluster_caching.md
│   │   ├── cluster_greedy.md
│   │   ├── cluster_heap.md
│   │   └── cluster_preprocessing.md
│   ├── ideas/                        # Layer 2 — individual idea files
│   │   ├── active/
│   │   ├── established/
│   │   ├── disputed/
│   │   ├── debunked/
│   │   └── archived/
│   ├── patterns/                     # Layer 2 — individual pattern files
│   │   ├── active/
│   │   └── confirmed/
│   ├── facts/                        # Global — flat directory, no lifecycle
```

```
subdirs
|   ├── observations/                # Raw, per agent-instance per generation
|   ├── research/                    # Research findings, per instance per gen
|   └── experiments/                 # Experimentator results, per gen per
instance
|       └── gen007/
|           └── experimentator_1/
|               ├── experiment_results.md
|               └── sandbox/         # Preserved test code and raw data
|
├── history/                         # Written only by orchestrator
|   ├── generations/                 # Per-generation snapshots
|   ├── score_progression.md
|   ├── solution_idea_map.md         # Many-to-many with aggregated stats
|   └── coverage_matrix.md           # Structured: which idea combos tried
|
├── briefs/                          # Written only by orchestrator from
Architect output
|   └── gen007/
|       ├── manifest.yaml            # Execution plan — orchestrator reads this
|       ├── manifest_reasoning.md    # Why these agents, for humans + analysis
|       ├── explore_1.md
|       ├── explore_2.md
|       ├── exploit_1.md
|       ├── genetic_1.md
|       ├── full_1.md
|       ├── research_1.md
|       └── experimentator_1.md
|
├── reports/                         # Written only by orchestrator from debrief
outputs
|   └── gen007/
|       ├── explore_1.md
|       ├── explore_2.md
|       ├── exploit_1.md
|       ├── genetic_1.md
|       ├── full_1.md
|       ├── research_1.md
|       ├── experimentator_1.md
|       ├── evaluator.md
|       └── system_critic.md
|
├── feedback/                        # Written only by orchestrator from analysis
agent outputs
|   ├── agent_gaps/                  # Per-gen synthesis of what agents lacked
|   ├── system_analysis/             # System Critic's pipeline analysis
|   ├── system_recommendations.md    # Rolling actionable list for user
|   ├── experiment_suggestions/      # System Critic's recommended experiments
|   └── consistency_reviews/         # Every 3rd generation
```

```
|
├── workspace/                        # Ephemeral agent work directories
|   ├── gen007_explore_1/             # Created by orchestrator before run
|   |   ├── prompt.md                 # Copied from agents/{type}.md
|   |   ├── brief.md                  # Copied from briefs/
|   |   └── output/                   # ONLY place this agent can write
|   |       ├── sol01.py
|   |       ├── sol02.py
|   |       └── observations.md
|   ├── gen007_experimentator_1/
|   |   ├── prompt.md
|   |   ├── brief.md
|   |   └── output/
|   |       ├── experiment_results.md
|   |       └── sandbox/              # All experiment code runs here
|   |           ├── test_cache.py
|   |           ├── test_no_cache.py
|   |           └── results_raw.csv
|   ├── gen007_evaluator/
|   |   ├── prompt.md
|   |   └── output/                   # Evaluator writes knowledge here
|   |       ├── new_ideas/
|   |       ├── updated_ideas/
|   |       ├── new_patterns/
|   |       ├── updated_clusters/
|   |       ├── solution_idea_map.md
|   |       ├── coverage_matrix.md
|   |       ├── generation_snapshot.md
|   |       └── evaluator_report.md
|   └── gen007_consistency_reviewer/
|       ├── prompt.md
|       └── output/
|           ├── state_of_affairs.md
|           ├── updated_ideas/
|           ├── updated_clusters/
|           └── consistency_review.md
|
├── user/                             # User configuration (read-only for agents)
|   ├── initial_ideas.md
|   ├── initial_facts.md
|   ├── interventions.md              # Auto-tracked user edit log
|   └── config.yaml
|
├── agents/                           # Agent prompt templates (read-only)
|   ├── architect.md
|   ├── explore.md
|   ├── exploit.md
|   ├── genetic.md
|   ├── full.md
|   ├── research.md
```

```
│   ├── research.md
│   ├── experimentator.md
│   ├── evaluator.md
│   ├── system_critic.md
│   └── consistency_review.md
│
├── orchestrator.py                  # Stateless loop: read files → launch agents
→ move outputs
└── CLAUDE.md                        # Project context for Claude Code sessions
```

9.3 Write Permissions Summary

The file system enforces a clear ownership model. Agents never write directly to shared directories. The orchestrator is the only process that writes to permanent locations.

Directory	Written by	Read by
`problem/`	User (setup)	All agents
`population/`	Orchestrator (moves from agent output)	All agents
`knowledge/`	Orchestrator (moves from Evaluator/ Reviewer output)	All agents
`history/`	Orchestrator (moves from Evaluator output)	All agents
`briefs/`	Orchestrator (moves from Architect output)	All agents, orchestrator reads manifest
`reports/`	Orchestrator (moves from debrief output)	All agents
`feedback/`	Orchestrator (moves from Critic/ Reviewer output)	All agents, user
`workspace/{agent}/output/`	That specific agent	Orchestrator (to move outputs)
`user/`	User	All agents, orchestrator
`agents/`	User (setup)	Orchestrator (copies to workspaces)

9.4 Workspaces

Each agent instance gets an ephemeral workspace created by the orchestrator before its run. The workspace contains a copy of its prompt template and brief. The agent has read access to the entire project file system (it can read from any directory) but writes only to `output/` within its workspace.

The Experimentator's workspace additionally contains a `output/sandbox/` directory where it runs all experiment code. This isolates test scripts, temporary files, and experiment artifacts.

After each session, the orchestrator moves the contents of `output/` to the appropriate permanent location and archives or deletes the workspace.

---

10. User Interaction

10.1 The User Never Blocks the System

Alpha Evolve runs autonomously. The user sets up the problem, starts the system, and optionally intervenes between generations by editing files. The system never waits for user input.

10.2 Before the First Run

The user creates: `problem/description.md`, `problem/constraints.md`, `problem/evaluate.py`, and test case files. Optionally, the user seeds knowledge: `user/initial_ideas.md` (implementation strategies to try) and `user/initial_facts.md` (known environment constraints). The user configures the system in `user/config.yaml`.

10.3 Between Generations

The user can:

- Read `knowledge/state_of_affairs.md` to see the system's compressed self-knowledge at a glance
- Read `knowledge/clusters/` to see current understanding of each approach family
- Read `feedback/system_recommendations.md` to see what the system thinks needs changing
- Read `feedback/system_analysis/` for detailed pipeline analysis
- Read `knowledge/experiments/` to see controlled experiment results
- Read any agent report for transparency
- Read any knowledge file to understand what the system believes
- Edit or create knowledge files directly (add an idea, correct a fact, debunk an insight)
- Edit the State of Affairs to correct the system's strategic understanding
- Edit a cluster summary to correct the system's understanding of an approach family
- Edit an agent's prompt template to adjust behavior
- Override a brief before the next generation starts
- Adjust config (enable/disable agents, change instance limits, change models)

All user edits are auto-detected by the orchestrator and logged in `user/interventions.md`.

10.4 After the Run

The user reads the final best solution, the State of Affairs (what did the system learn overall), the knowledge base (detailed learnings), the score progression (how did evolution proceed), system recommendations (what should change), experiment results (what was tested), and consistency reviews (is the knowledge coherent).

---

11. Cold Start — Generation 1

Generation 1 has no population, no history, and minimal knowledge. Most agent roles do not fully apply yet. The knowledge hierarchy has not been built yet.

What runs: Explore (1-2 instances producing diverse initial solutions), Full Agent (1 instance, unconstrained attempt), Research (1 instance, investigating known approaches). Exploit is skipped or given minimal work (nothing good to refine). Genetic is skipped (no parents to cross). Experimentator is skipped (no questions to test yet).

What the Architect does: Writes a simple manifest and briefs. "Try different approaches." "Research known algorithms for this problem type." No State of Affairs or clusters exist yet, so the Architect works directly from the problem description and any user-provided initial knowledge.

Knowledge bootstrap: After generation 1's Evaluator pass, the system has its first Layer 2 files (ideas, patterns, facts extracted from generation 1's results), its first Layer 1 files (initial topic clusters created by the Evaluator), the first coverage matrix, and the first State of Affairs (Layer 0, written as a simple initial version by the Evaluator). The Consistency Reviewer writes the first full State of Affairs at generation 3.

By generation 2, there is a leader to refine (Exploit activates), research findings to incorporate, and a nascent knowledge hierarchy to orient agents. By generation 3, enough diversity exists for Genetic to start crossing. The Consistency Reviewer runs its first audit and writes the first full State of Affairs. The Architect may begin launching Experimentator instances if open questions have emerged. The system is fully operational.

Rich cold start alternative: If the user provides detailed `initial_ideas.md` and `initial_facts.md`, the system can start more targeted from generation 1. The Evaluator will incorporate user-provided ideas into the initial cluster structure.

---

12. Configuration

yaml

```yaml
# user/config.yaml

generations: 20
target_score: 0.95
evaluation_timeout: 30

agents:
  explore:
    enabled: true
    max_instances: 3
  exploit:
    enabled: true
    max_instances: 3
  genetic:
    enabled: true
    max_instances: 3
    parents_per_instance: 2
  full:
    enabled: true
    max_instances: 2
  research:
    enabled: true
    max_instances: 2
  experimentator:
    enabled: true
    max_instances: 3

analysis:
  evaluator:
    enabled: true
    model: opus
  system_critic:
    enabled: true
    model: opus

consistency_review_interval: 3
emergency_review_on_strategic_shift: true

knowledge_hierarchy:
  state_of_affairs_max_tokens: 1500
  cluster_max_tokens: 400
  max_clusters: 20
  merge_threshold: 0.7  # similarity above which clusters should merge

idea_limits:
  max_ideas: 100
  threshold_30: good
```

```
  threshold_50: very_good
  threshold_100: revolutionary

staleness_threshold: 5

default_model: sonnet
architect_model: opus
debrief_model: haiku

max_parallel_sessions: 10
```

---

13. What Makes This Different

The original AlphaEvolve is a single LLM loop that generates, evaluates, and iterates. It is powerful because it grounds evolution in code execution, uses rich context, and operates at scale with thousands of LLM samples.

This system inherits those principles but restructures them:

Specialization. Instead of one LLM wearing all hats, specialized agents do what they do best. Explore diverges. Exploit refines. Genetic combines. Research investigates. The Experimentator runs controlled tests. The Evaluator extracts knowledge. The System Critic debugs the pipeline. Each agent's prompt is optimized for its cognitive task.

Hierarchical accumulated knowledge. Instead of a program database that stores solutions and scores, this system maintains a three-layer knowledge hierarchy: a single State of Affairs document for strategic orientation, topic cluster summaries for thematic understanding, and individual idea/pattern/fact files with full provenance and evidence. Knowledge accumulates, evolves, gets confirmed or debunked, and informs future generations — without overwhelming agent context windows as generations grow. An agent in generation 50 reads the same compact State of Affairs as an agent in generation 3, with the option to drill deeper when relevant.

Coverage tracking. The State of Affairs includes a coverage map backed by a structured coverage matrix. The matrix is hard data — which idea combinations have been tried, how many times, best score per combination. The narrative coverage map in the State of Affairs is cross-checked against this data. Together they prevent the system from retreading explored ground even over very long runs.

Controlled experimentation. The Experimentator agent runs targeted tests that answer specific questions, producing high-confidence knowledge rather than score-chasing solutions. This fills a gap that no other agent type addresses: the difference between "we tried X and it scored 0.7" (incidental observation) and "we tested X with and without Y and measured the difference" (controlled experiment).

Files as single source of truth. All system state lives in the file system. The orchestrator is a stateless Python loop that reads files, launches agents, and moves outputs. No state is held in memory. If the orchestrator crashes, it reconstructs its position from the files. The Architect controls strategy by writing a manifest that the orchestrator executes literally. Every component communicates through files.

Read broadly, write narrowly. All agents can read the entire project. No agent can write outside its own output directory. The orchestrator is the only process that moves files to permanent locations. This prevents agents from corrupting each other's work or the knowledge base, while preserving the ability to browse broadly for context.

Self-awareness. Through the debrief system and the System Critic, the system identifies its own limitations and reports them. Agents say what they lacked. The System Critic identifies missing capabilities. The Consistency Reviewer audits old beliefs and rewrites the system's strategic self-knowledge. The system doesn't just optimize solutions — it optimizes itself.

Human-in-the-loop without blocking. The user can intervene at any point by editing files — adding knowledge, correcting facts, adjusting the State of Affairs, editing cluster summaries, adjusting prompts, changing configuration. But the system never waits. It runs autonomously and the user observes, assists, and steers when they choose to.

Parallel depth. Multiple instances of each agent type, each iterating internally. A single generation might involve 10+ parallel Claude Code sessions, each trying multiple approaches, producing 15-25 evaluated solutions plus controlled experiments. Each instance has a distinct mission from the Architect, informed by the coverage map to avoid redundant exploration.

The system is designed so that every component earns its place. If an agent type is not contributing, the Architect can set its instance count to zero in the manifest. If the knowledge system is too noisy, the idea limits tighten. If the debriefs aren't useful, the System Critic will flag it. If the cluster summaries drift from reality, the Consistency Reviewer corrects them. The architecture is self-correcting by design.